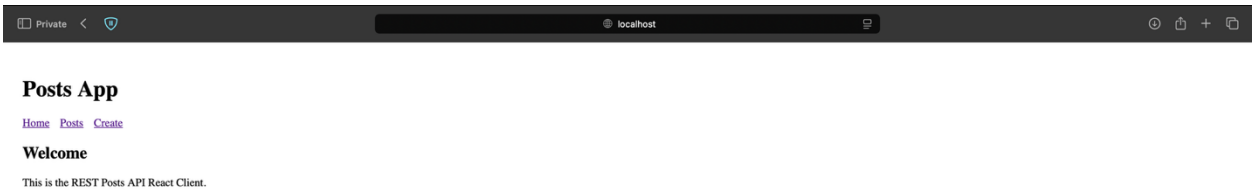
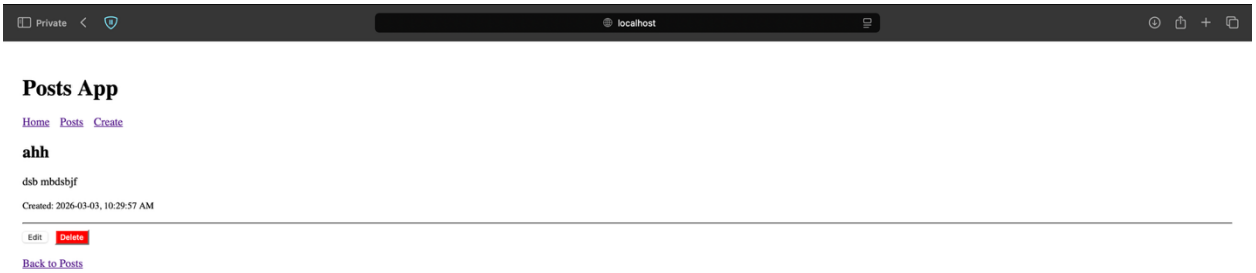
REPORT

6.1 Screenshots

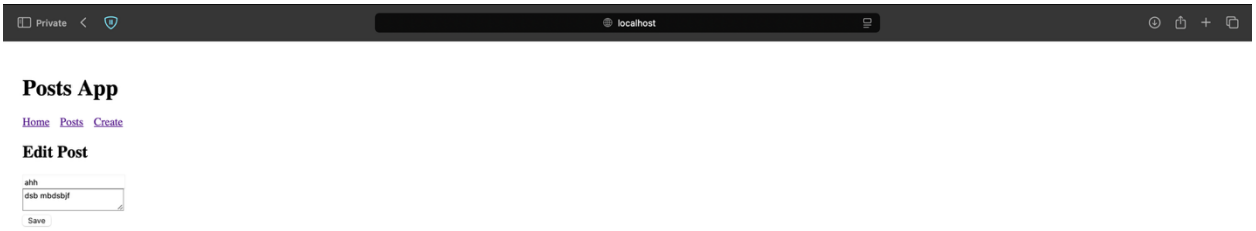
Landing Page:



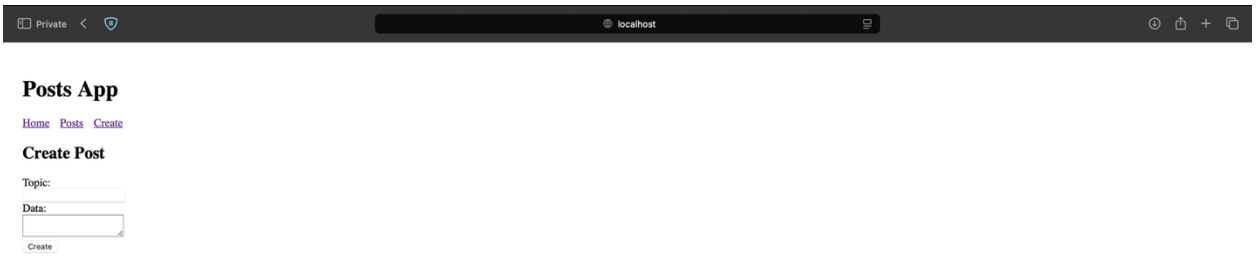
Post Details Page



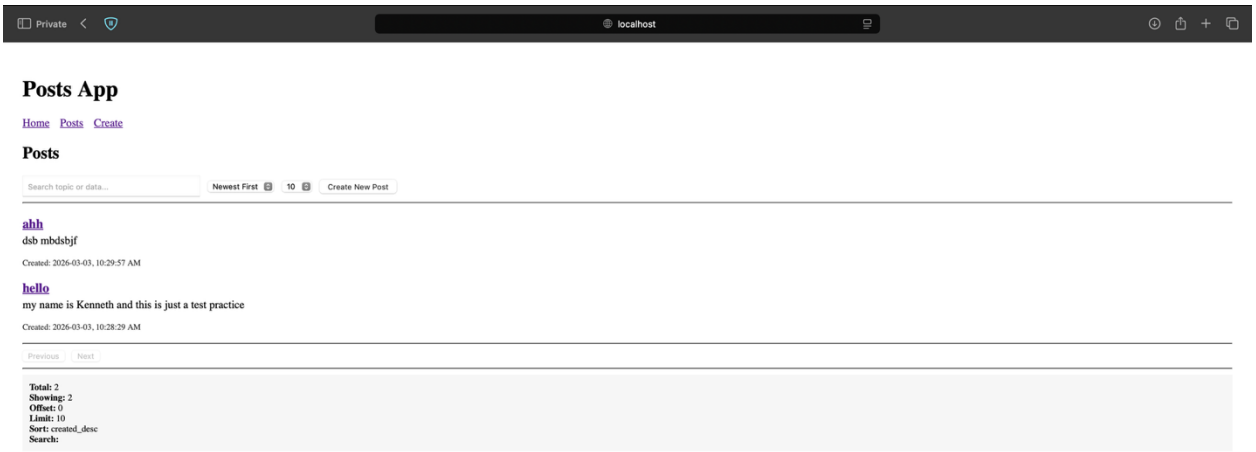
Edit Post page



Create Post Page



Posts list page



6.2 Overall Testing Approach

I tested both the backend API and the frontend interface to ensure correctness, validation, and proper error handling.

Backend Testing

- I manually tested all endpoints using the browser and fetch-based frontend calls.
- I verified that GET /api/posts returned **200 OK** and included correct pagination metadata.
- I tested POST /api/posts to confirm it returned **201 Created** and inserted records correctly into the database.
- I verified that PUT /api/posts/:id returned **200 OK** and correctly replaced both the topic and data fields.
- I tested PATCH /api/posts/:id to ensure partial updates worked and returned **200 OK**.

- I confirmed that DELETE /api/posts/:id returned **204 No Content** and removed the record.
- I tested invalid inputs (e.g., missing topic or data) and verified the API returned **400 Bad Request**.
- I tested non-existent IDs and confirmed the API returned **404 Not Found**.

Frontend Testing

- I verified that navigation links between pages (Home, Posts, Create, Edit) worked correctly.
- I tested creating a new post and confirmed it appeared in the posts list.
- I tested editing a post and confirmed changes were reflected on the details page.
- I tested deleting a post and confirmed it was removed from the list.
- I verified that error messages appeared when invalid input was submitted.
- I confirmed that search, sorting, and pagination controls functioned as expected.

6.3 Load Testing (using loadtest)

I ran the following command to test the performance of the list endpoint:

npx loadtest -c 10 -n 1000 http://localhost/api/posts?limit=20&offset=0

Results

- Total requests: 1000
- Concurrent clients: 50
- Total errors: 0
- Total time: 0.666 seconds
- Mean latency: 31.9 ms
- Effective requests per second (RPS): 1502

```
tomez@Olisas-MacBook-Pro assignment5 % npx loadtest -c 10 -n 1000
"http://localhost/api/posts?limit=20&offset=0"
(node:8595) [DEP0060] DeprecationWarning: The `util._extend` API is
deprecated. Please use Object.assign() instead.
(Use `node --trace-deprecation ...` to show where the warning was
created)
(node:8596) [DEP0060] DeprecationWarning: The `util._extend` API is
deprecated. Please use Object.assign() instead.
(Use `node --trace-deprecation ...` to show where the warning was
created)
(node:8598) [DEP0060] DeprecationWarning: The `util._extend` API is
deprecated. Please use Object.assign() instead.
(Use `node --trace-deprecation ...` to show where the warning was
created)
(node:8597) [DEP0060] DeprecationWarning: The `util._extend` API is
deprecated. Please use Object.assign() instead.
(Use `node --trace-deprecation ...` to show where the warning was
created)
(node:8599) [DEP0060] DeprecationWarning: The `util._extend` API is
deprecated. Please use Object.assign() instead.
(Use `node --trace-deprecation ...` to show where the warning was
created)
(node:8600) [DEP0060] DeprecationWarning: The `util._extend` API is
deprecated. Please use Object.assign() instead.
(Use `node --trace-deprecation ...` to show where the warning was
created)

Target URL:          http://localhost/api/posts?limit=20&offset=0
Max requests:        1000
Concurrent clients:  50
Running on cores:    5
Agent:               none

Completed requests:  1000
Total errors:        0
Total time:          0.666 s
Mean latency:        31.9 ms
Effective rps:       1502

Percentage of requests served within a certain time
  50%      30 ms
  90%      35 ms
  95%      47 ms
  99%      80 ms
 100%      90 ms (longest request)
tomez@Olisas-MacBook-Pro assignment5 %
```

Latency Breakdown

- 50% of requests completed within 30 ms
- 90% within 35 ms
- 95% within 47 ms
- 99% within 80 ms
- Maximum latency: 90 ms

Analysis

The API handled 1000 requests with zero errors, demonstrating stable performance under concurrent load. The average latency was approximately 32 ms, which indicates fast response times. Even at higher percentiles (95–99%), latency remained under 80 ms, showing consistent performance. Overall, the list endpoint performs efficiently under moderate concurrency.